

Due: Sunday, 24 May 2026, 11:59 PM

Vous devez implanter en C++, les classes formant l'arbre d'héritage de Shape vue en TD.

Ouvrir une fenêtre

À titre d'introduction à la programmation d'interfaces graphiques (qui sera vue dans la ressource R2.02), vous devrez utiliser la mini-bibliothèque [EZDraw++](#) afin d'ouvrir une simple fenêtre à l'écran. Ce sera également un premier travail de découverte d'une documentation technique (assez réduite et en français) mais également l'occasion pour vous de mettre en pratique les notions d'héritage et de polymorphisme (car elles sont nécessaires ne serait-ce que pour ouvrir une simple fenêtre).

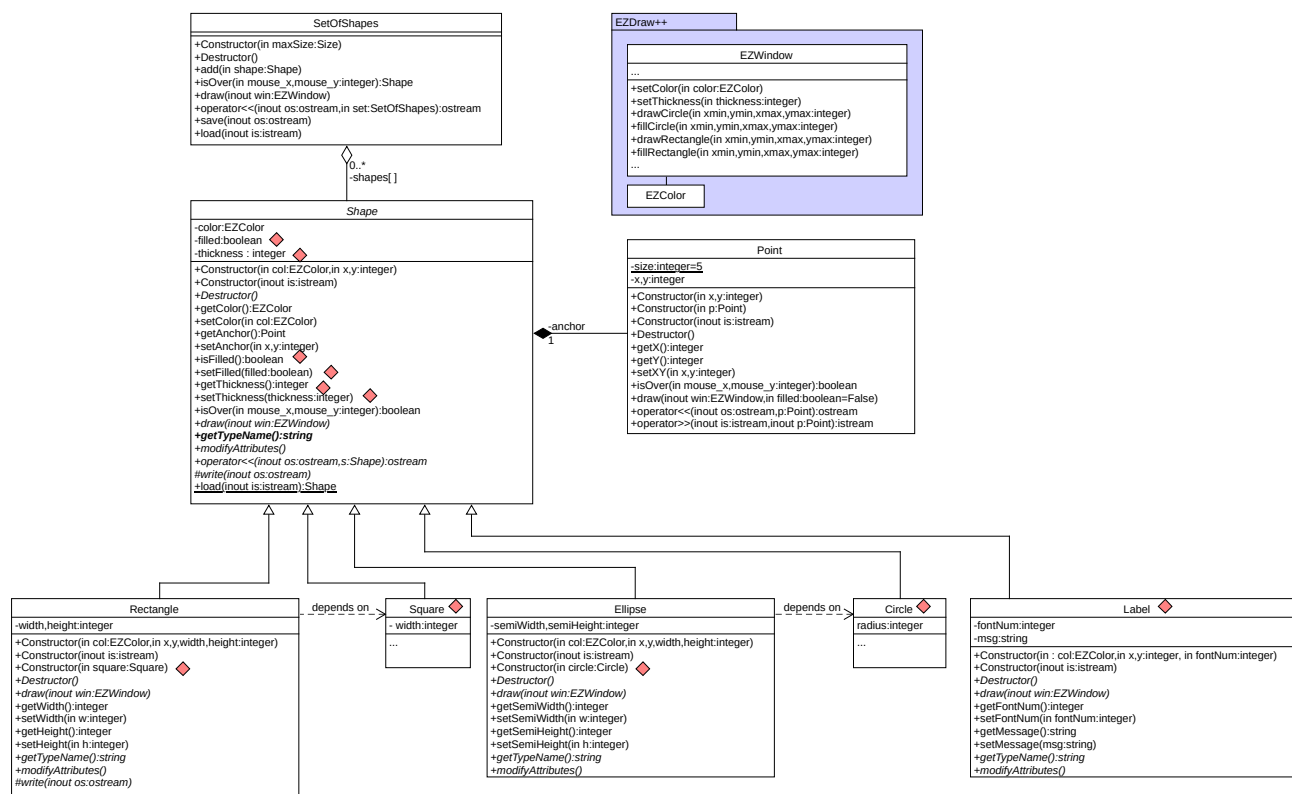
Chez vous et pas pendant le TP, vous lirez le tutoriel sur la page Web d'EZDraw++ et, en parallèle, vous étudierez les exemples fournis. Votre objectif est de savoir ouvrir une fenêtre et de savoir y dessiner quelques formes géométriques.

Implanter les classes vues en TD

En salle de TP, vous implanterez les classes vues en TD : Point, Shape, Rectangle et SetOfShapes.

Vous devrez également créer les classes Square et Circle telles qu'expliquées en TD. Vous devrez rajouter dans Rectangle un constructeur qui permet de créer un Rectangle à l'identique d'une instance de Square (donc ayant les mêmes valeurs de largeur et hauteur) puis, sur le même principe, un constructeur de Ellipse qui permet d'en créer une à l'identique d'un Circle reçu en paramètre.

Le diagramme de classe UML complété qui correspond à ce travail est consultable ci-dessous avec des marques rouges en forme de diamant pour indiquer les modifications par rapport à celui de la feuille de TD.



Récupérez également les fichiers présents dans l'archive suivante : [\[Shapes.zip\]](#). Vous y trouverez un main.cpp et une classe MyWindow toute prête à faire fonctionner vos classes ainsi que trois Makefiles (Linux et deux pour Windows).

Vous trouverez le reste des informations au sujet d'EZDraw++ dans le tutoriel, en particulier le chapitre qui concerne le dessin et la couleur. Sachez que, comme vous rentrez dans le cœur de ce pour quoi le langage C++ et la P.O.O. ont été faits (les gros projets), ce TP est un peu austère : vous allez devoir coder *beaucoup* pour finalement *peu* de résultat visuel. L'intérêt de tout cela est donc avant tout d'apprendre et de comprendre l'architecture Orientée Objet de l'exercice.

?

Précisions sur la classe Point

Dans l'implantation de la méthode « void Point::draw(EZWindow& w, bool isActive=false) const », vous dessinerez un petit carré qui s'étend de « size » pixels de part et d'autre des coordonnées du point. Pour « size » valant 3 comme vu en TD, le carré fait donc $3+1+3 = 7$ pixels de haut et de large.

Si le paramètre « isActive » est à Vrai, vous devrez dessiner le carré plein et en rouge, sinon uniquement son bord et en noir. Cela permettra d'identifier visuellement les points et les formes sélectionnées par l'utilisateur.

Précisions sur la classe Shape

On rappelle que cette classe doit être complétée par les attributs et méthodes suivants :

- un booléen « filled » qui indique si la forme est pleine ou non ;
- deux accesseurs publics liés à cet attribut : « bool isFilled() const » et « void setFilled(bool new_value=false) » ;
- un entier « thickness » privé qui indique l'épaisseur avec laquelle le tracé extérieur de la forme doit être effectué et qui, par défaut, vaudra 1 ;
- deux accesseurs public liés à cet attribut : « int getThickness() const » et « void setThickness(int _thickness) ».

On rappelle ce qui a été vu en TD :

- on choisit de dire qu'être au dessus d'une forme se limite à être au dessus de son ancre, autrement dit la fonction Shape::isOver() doit simplement faire appel à Point::isOver() sur l'attribut anchor.

Précisions sur la classe Rectangle

Dans l'implantation de la méthode « void Rectangle::draw(EZWindow& w) const », vous devrez dessiner soit un rectangle plein si l'attribut « Shape::filled » est Vrai et un rectangle vide sinon.

Vous devrez faire pareil pour toutes les autres classes pouvant être dessinées de manière vide ou pleine, c'est-à-dire au moins les classes Square, Ellipse et Circle.

Précisions sur la classe SetOfShapes

Dans l'implantation de la méthode « void SetOfShapes::draw(EZWindow& w) const », pour chaque forme contenue dans l'ensemble, vous dessinerez d'abord cette forme puis son ancre. Lors du dessin de l'ancre, vous transmettez à la méthode de dessin du point la valeur de l'attribut « selected » de la forme.

La méthode « SetOfShapes::isOver(int mouse_x, int mouse_y) const » doit vous renvoyer la forme qui est actuellement survolée (s'il y en a une qui est survolée, c'est-à-dire que l'appel de sa fonction « Shape::isOver(int mouse_x, int mouse_y » renvoie true) ou bien qu'aucune forme n'est actuellement sous la souris. Sachant qu'il n'est pas possible de renvoyer littéralement une instance de Shape (puisque c'est une classe Abstraite) et qu'il faut aussi pouvoir renvoyer un résultat équivalent à « aucune forme », la meilleure solution consiste à renvoyer l'adresse de la forme survolée ou, sinon, nullptr.

Modifier les attributs d'une forme

Sur le diagramme UML de classes, vous verrez qu'il y a, dans Shape et toutes ses classes dérivées, une méthode virtuelle « modifyAttributes() ». Son rôle est de permettre à l'utilisateur de consulter et modifier les valeurs des attributs de cette classe. Pour faire cela simplement, je vous propose d'utiliser un menu textuel sur la console. Chaque niveau d'héritage apportant son lot d'attributs, il faut faire non seulement une version par classe mais également faire en sorte qu'un niveau soit capable d'appeler le niveau dont il hérite.

On aura donc pour Shape quelque chose dans le style de la fonction fournie dans le fichier [Shape_modifyAttributes.cpp] de l'archive ZIP. Et pour Rectangle, on aura [Rectangle_modifyAttributes.cpp]. Incorporez le contenu de ces fichiers à vos classes correspondantes. Puis, à vous de compléter les autres classes sur le même principe.

N'oubliez pas un détail important : lorsque le programme attend que l'utilisateur tape une réponse sur la console et la valide par un appui sur la touche « Entrée », le traitement des événements et donc le rafraîchissement de la fenêtre sont bloqués !

Au delà du sujet...

Si vous souhaitez aller un peu plus loin dans l'aspect théorique, je vous invite à lire la page Web de Wikipédia au sujet du problème d'héritage évoqué en TD : https://en.wikipedia.org/wiki/Circle-ellipse_problem.

Pour le côté programmation, il est maintenant très simple (mais un peu long pour le nombre de séances de TP), de rajouter plusieurs fonctionnalités (très chouettes) sur votre programme :

- ajouter de nouvelles classes de formes : un message textuel (classe Label), un objet image (classe Image, qui utilise un JPEG ou PNG sur le disque), etc. ;
- supprimer la forme sélectionnée (touche Suppr dans MyWindow.cpp qui nécessite d'avoir une méthode SetOfShape::remove(Shape*) qui supprime la forme indiquée de la liste puis la détruit) ;
- utiliser la souris pour modifier des formes existantes (toujours avec un mécanisme de sélection) ;
- etc.

Si vous réalisez ces fonctions supplémentaires sur votre temps libre, montrez-les (à moi et aux autres étudiants) en TP.

Éliminer la dépendance de l'implantation de Shape::load(istream&) aux déclarations des sous-classes de Shape

Cette partie concerne les étudiants les plus curieux et motivés : il s'agit d'éliminer le problème de conception qui a été expliqué en TD.

L'idée est de permettre à la méthode Shape::load(istream&) de trouver une association entre une chaîne de caractère, par exemple "Rectangle" ou "Ellipse" avec un pointeur pour chacune vers une fonction membre des classes de même nom qui réalise la construction d'une instance de cette classe à partir d'une référence sur istream.

Les modifications suivantes sont nécessaires :

- ajouter à chaque classe fille XXX de Shape une méthode de classe (statique) « Shape *XXX::create(istream& is) » qui fait simplement ce qu'on faisait dans le if() initial : « { return new XXX(is); } ».
- ajouter à la classe Shape un attribut de classe (statique) qui est un tableau associatif (std::map<>) qui lie une chaîne (std::string) à un pointeur sur le type de la fonction précédente, c'est-à-dire « Shape* (*function)(istream&) ».
- généraliser dans Shape::load(istream&) la recherche en utilisant le map précédent : lorsqu'on trouve la chaîne, on déclenche la fonction associée pour créer la forme.
- pour finir, il faut penser, au démarrage du programme et une seule fois, à insérer pour chaque classe fille de Shape, son nom associé à l'adresse de sa fonction create dans le map.

Faire tout cela proprement (sans répétition de code) peut vous amener à écrire un patron et des [lambdas](#)...

Rendu

Lisez attentivement *toutes* les consignes et respectez-les ! Vous devez rendre l'ensemble des fichiers sources C++ (.hpp et .cpp), y-compris les fichiers qui vous ont été fournis (EZDraw++ et MyWindow) afin que le projet soit complet et compilable (idéalement avec un Makefile). Pour faciliter la correction, laissez l'ensemble de ces fichiers à la racine d'un unique répertoire.

Vos classes seront testées une par une (tests unitaires) puis ensemble (tests d'intégration). Il vaut donc mieux avoir seulement quelques classes qui marchent parfaitement (pour gagner une partie de points), que d'essayer de faire un peu de tout sans rien finir donc sans que rien ne compile et marche du tout (n'avoir aucun point).

Indiquez-moi si vous avez besoin d'une précision supplémentaire.

Add submission

Submission status

Submission status	No submissions have been made yet
Grading status	Not graded
Time remaining	14 hours remaining
Last modified	-
Submission comments	> Comments (0)

◀ TP n°4, 5 et 6 : liste (simplement) chaînée

Jump to activity

Complément à « Formes géométriques et polymorphisme » : supprimer la dépendance de Shape à ses classes dérivées ►